

Guide des modules 4 & 5

Machine Learning & Deep Learning — Projet RetailSense AI

Ce guide accompagne les étudiants dans les modules de **Machine Learning** (chapitre 6) et de **Deep Learning** (chapitre 7). Pour **chaque modèle**, il précise : ce qu'on attend de lui, ses entrées, les caractéristiques de la base utilisées pour l'entraînement, ses sorties, et les paramètres et hyperparamètres à régler.

Comment lire chaque fiche

Rubrique	Signification
Ce qu'on attend	l'objectif du modèle et la tâche à résoudre.
Entrées	ce qu'on donne au modèle au moment de la prédiction.
Caractéristiques utilisées	les variables de la base (ou le type de donnée) servant à l'entraînement.
Sorties	ce que le modèle produit.
Hyperparamètres à régler	les réglages fixes par vous, à ajuster pour de bons résultats.
Appris automatiquement	les paramètres estimés pendant l'entraînement (vous ne les fixez pas).

Source de données commune

Tous les modèles s'appuient sur la **base SQLite** construite en phase 1 et **préparée en phase 2** (module *retailsense_features.py*), qui fournit deux tables prêtes : une **table au niveau commande** (variables + cibles) et une **table RFM au niveau client**. Les modules image, texte et graphe utilisent en plus des données dédiées (voir leurs fiches).

Rappel : les caracteristiques disponibles dans la base

Table au niveau commande (une ligne par commande)

Caracteristique	Description	Source dans la base
total_price	Somme des prix des articles	order_items.price
total_freight	Somme des frais de port	order_items.freight_value
total_weight	Poids total de la commande	products.product_weight_g
n_items	Nombre d'articles	order_items
max_installments	Nombre max de mensualites	order_payments.payment_installments
payment_value	Montant total paye	order_payments.payment_value
delivery_days	Delai de livraison (jours)	orders (livraison - achat)
delay_days	Retard vs date estimee (jours)	orders (livraison - estimee)
late	Livraison en retard (0/1)	derive de delay_days
purchase_month / dow	Mois et jour de la semaine d'achat	orders.order_purchase_timestamp
main_category	Categorie principale (en anglais)	products + traduction
payment_type	Moyen de paiement	order_payments.payment_type
customer_state	Etat (region) du client	customers.customer_state

Cibles a predire

Cible	Definition	Type de tache
bad_review (0/1)	Avis negatif : review_score <= 2	Classification (modules ML/MLP)
delivery_days	Delai de livraison en jours	Regression

Autres donnees (par module)

Donnee	Contenu	Module concerne
RFM par client	recency, frequency, monetary (par customer_unique_id)	Segmentation, GAN
Serie temporelle	Nombre de commandes par jour	LSTM (prevision)
Texte des avis	order_reviews.review_comment_message + note	Transformer (sentiment)
Images produits	Images de catalogue (Fashion-MNIST en substitut)	CNN
Graphe clients-produits	Achats = liens (order_items + orders + customers)	GNN (recommandation)

Partie A - Module 4 : Machine Learning

Trois familles : la **segmentation** (non supervisee), la **classification** (5 algorithmes sur la meme cible, pour les comparer) et la **regression**. Pour les modeles a base d'arbres, la standardisation n'est pas necessaire ; pour la regression logistique, elle est recommandee.

1. Segmentation - K-means

Regrouper les clients en k segments homogenes pour cibler les actions marketing.

Ce qu'on attend	Constituer des groupes de clients au comportement d'achat similaire.
Entrees	Matrice RFM d'un client (3 valeurs), standardisee.
Caracteristiques utilisees	recency, frequency, monetary (log sur frequency et monetary, puis standardisation obligatoire).
Sorties	Un numero de segment par client + les centres (profils moyens) de chaque groupe.
Hyperparametres a regler	n_clusters (k) choisi via silhouette + methode du coude ; n_init (=10) ; standardisation ; random_state .
Appris automatiquement	Les positions des centroides (centres des clusters).

2. Segmentation - DBSCAN

Detecter des groupes de forme quelconque et isoler les clients atypiques, sans fixer k.

Ce qu'on attend	Trouver des clusters par densite et reperer les points isoles (comportements rares).
Entrees	Meme matrice RFM standardisee.
Caracteristiques utilisees	recency, frequency, monetary standardisees.
Sorties	Un numero de cluster par client ; le label -1 designe les points atypiques (bruit).
Hyperparametres a regler	eps (rayon de voisinage) choisi via la courbe des k-distances ; min_samples (~10) ; metrique de distance.
Appris automatiquement	L'affectation des points aux clusters (pas de centre explicite).

3. Classification - Regression logistique

Modele lineaire interpretable ; sert de reference aux autres.

Ce qu'on attend	Predire si une commande recevra un avis negatif.
Entrees	Vecteur des caracteristiques de la commande (numeriques + categorielles encodees en one-hot). La standardisation des numeriques est recommandee.
Caracteristiques utilisees	total_price, total_freight, total_weight, n_items, max_installments, payment_value, delivery_days, delay_days, purchase_month/dow, main_category, payment_type, customer_state.
Sorties	Probabilite d'avis negatif (0 a 1), puis classe 0/1 selon un seuil (0,5 par default).
Hyperparametres a regler	C (force de regularisation), penalty (l1/l2), class_weight='balanced' (desequilibre), solver , max_iter , seuil de decision.
Appris automatiquement	Les coefficients (poids) de chaque variable et le biais.

4. Classification - Arbre de decision

Modele sous forme de regles lisibles ; base des methodes d'ensemble.

Ce qu'on attend	Predire l'avis negatif avec un modele explicable.
Entrees	Vecteur des caracteristiques de la commande (numeriques + categorielles encodees en one-hot). Pas besoin de standardiser.
Caracteristiques utilisees	total_price, total_freight, total_weight, n_items, max_installments, payment_value, delivery_days, delay_days, purchase_month/dow, main_category, payment_type, customer_state.
Sorties	Probabilite d'avis negatif (0 a 1), puis classe 0/1 selon un seuil (0,5 par default). Regles de decision visualisables.
Hyperparametres a regler	max_depth , min_samples_leaf , min_samples_split, criterion (gini/entropy), class_weight, ccp_alpha (elagage).
Appris automatiquement	La structure de l'arbre : variables et seuils de coupure.

5. Classification - Random Forest

Agregation de nombreux arbres (bagging) : robuste et performant.

Ce qu'on attend	Ameliorer la stabilite et la performance de l'arbre seul.
Entrees	Vecteur des caracteristiques de la commande (numeriques + categorielles encodees en one-hot).
Caracteristiques utilisees	total_price, total_freight, total_weight, n_items, max_installments, payment_value, delivery_days, delay_days, purchase_month/dow, main_category, payment_type, customer_state.
Sorties	Probabilite d'avis negatif (0 a 1), puis classe 0/1 selon un seuil (0,5 par default). + importance des variables.
Hyperparametres a regler	n_estimators (~300), max_features ('sqrt'), max_depth , min_samples_leaf , class_weight='balanced_subsample' , bootstrap.
Appris automatiquement	L'ensemble des arbres (chaque arbre est appris sur un echantillon).

6. Classification - AdaBoost

Boosting : combine des modeles faibles en pondérant les erreurs.

Ce qu'on attend	Predire l'avis negatif en corrigeant progressivement les erreurs.
Entrees	Vecteur des caracteristiques de la commande (numeriques + categorielles encodees en one-hot).
Caracteristiques utilisees	total_price, total_freight, total_weight, n_items, max_installments, payment_value, delivery_days, delay_days, purchase_month/dow, main_category, payment_type, customer_state.
Sorties	Probabilite d'avis negatif (0 a 1), puis classe 0/1 selon un seuil (0,5 par default).
Hyperparametres a regler	n_estimators , learning_rate , profondeur de l'estimateur de base (souche) ; desequilibre gere via sample_weight .
Appris automatiquement	Les poids attribues a chaque estimateur et a chaque exemple.

7. Classification - Gradient Boosting

Arbres construits en sequence pour corriger l'erreur residuelle ; souvent le meilleur sur tabulaire.

Ce qu'on attend	Obtenir la meilleure performance de classification sur donnees tabulaires.
Entrees	Vecteur des caracteristiques de la commande (numeriques + categorielles encodees en one-hot).
Caracteristiques utilisees	total_price, total_freight, total_weight, n_items, max_installments, payment_value, delivery_days, delay_days, purchase_month/dow, main_category, payment_type, customer_state.
Sorties	Probabilite d'avis negatif (0 a 1), puis classe 0/1 selon un seuil (0,5 par default). + importance des variables.
Hyperparametres a regler	learning_rate (~0,05) et n_estimators (~300) — a regler ensemble ; max_depth (~3), subsample (~0,9), min_samples_leaf.
Appris automatiquement	Les arbres successifs (chacun corrige le precedent).

8. Regression (lineaire + ensembles)

Predire une valeur continue : ici le delai de livraison (adaptable a un prix / une demande).

Ce qu'on attend	Estimer le delai de livraison en jours.
Entrees	Vecteur des caracteristiques de la commande (numeriques + categorielles encodees).
Caracteristiques utilisees	total_price, total_freight, total_weight, n_items, payment_value, purchase_month, main_category, customer_state (sans delay_days/late qui derivent de la cible).
Sorties	Une valeur predite en jours ; les residus pour le diagnostic.
Hyperparametres a regler	Lineaire : aucun (reference). RandomForestRegressor : n_estimators, min_samples_leaf. GradientBoostingRegressor : learning_rate, n_estimators, max_depth, subsample.
Appris automatiquement	Les coefficients (lineaire) ou les arbres (ensembles).

Partie B - Module 5 : Deep Learning

Chaque architecture est adaptee a un type de donnee : tabulaire (MLP), image (CNN), sequence temporelle (LSTM), texte (Transformer), plus l'autoencodeur, le GAN et le GNN. Reglages transversaux communs : **optimiseur** (Adam par default ; AdamW ; RMSProp), **taux d'apprentissage**, **taille de lot** (batch_size), **nombre d'epoques**, et les **callbacks** (EarlyStopping, ReduceLROnPlateau, ModelCheckpoint).

1. MLP - perceptron multicouche (tabulaire)

Reseau dense sur donnees tabulaires, a comparer avec les modeles de la phase 4.

Ce qu'on attend	Predire l'avis negatif a partir des variables de la commande.
Entrees	Vecteur numerique dense (memes caracteristiques que la classification ML, imputees + standardisees + one-hot).
Caracteristiques utilisees	total_price, total_freight, total_weight, n_items, max_installments, payment_value, delivery_days, delay_days, purchase_month/dow, main_category, payment_type, customer_state.
Sorties	Probabilite d'avis negatif (couche de sortie sigmoide).
Hyperparametres a regler	Nombre et largeur des couches (128 puis 64), activation (ReLU), Dropout (0,3), L2 (1e-4), BatchNorm , Adam + learning_rate (1e-3), batch_size (64), epoques + EarlyStopping , class_weight.
Appris automatiquement	Les poids et biais de tous les neurones.

2. CNN - reseau de convolution (images produits)

Classer automatiquement les produits a partir de leur image.

Ce qu'on attend	Predire la categorie/type d'un produit depuis une image.
Entrees	Images sous forme de tenseurs (hauteur x largeur x canaux), normalisees. Ici Fashion-MNIST (28x28x1) en substitut, sinon un dossier d'images produits.
Caracteristiques utilisees	Les pixels bruts : le reseau apprend lui-meme les caracteristiques visuelles. Etiquette = classe du produit.
Sorties	Probabilites par classe (softmax), donc la categorie predite.
Hyperparametres a regler	Nombre de blocs Conv, nb de filtres (32 puis 64), taille de noyau (3x3), pooling, Dropout (0,25/0,4), augmentation de donnees , optimiseur/learning_rate, batch_size, epoques + callbacks ; (bonus) transfert d'apprentissage .
Appris automatiquement	Les filtres de convolution et les poids des couches denses.

3. RNN / LSTM (prevision de la demande)

Prevoir la demande future a partir de l'historique des commandes.

Ce qu'on attend	Prevoir le nombre de commandes du jour suivant.
Entrees	Sequences glissantes de la serie journaliere (fenetre de look_back pas x 1 variable), mises a l'echelle.
Caracteristiques utilisees	Serie temporelle du nombre de commandes par jour (derivee de order_purchase_timestamp). Variables exogenes possibles.
Sorties	La valeur prevue pour le pas suivant (regression), comparee a une reference naive.
Hyperparametres a regler	look_back (14), nb d'unites LSTM (64), nb de couches, dropout/recurrent_dropout, optimiseur/learning_rate, batch_size, epoques + EarlyStopping ; decoupage chronologique et scaler ajuste sur l'entrainement.
Appris automatiquement	Les poids des portes (entree, oubli, sortie) du LSTM.

4. Transformer (analyse de sentiment des avis)

Classer le sentiment des commentaires clients (positif / negatif).

Ce qu'on attend	Determiner si un avis est positif ou negatif.
Entrees	Texte de l'avis vectorise (sequences d'entiers de longueur fixe).
Caracteristiques utilisees	review_comment_message ; etiquette = positif (note >= 4) / negatif (note <= 2), les neutres (3) sont ecartes.
Sorties	Probabilite de sentiment positif (sigmoide).
Hyperparametres a regler	Taille du vocabulaire , longueur de sequence , dimension d'embedding (64), nb de tetes d'attention (2), taille du FFN, Dropout, LayerNorm, optimiseur/learning_rate, batch_size, epoques ; (production) affiner un modele pre-entraine multilingue.
Appris automatiquement	Les embeddings, les poids d'attention et le reseau feed-forward.

5. Autoencodeur (detection d'anomalies / fraude)

Reperer des commandes atypiques sans etiquettes, via l'erreur de reconstruction.

Ce qu'on attend	Detecter les commandes anormales (fraude, incidents).
Entrees	Vecteur numerique standardise ; entrainement sur les commandes normales uniquement.
Caracteristiques utilisees	total_price, total_freight, total_weight, n_items, max_installments, payment_value, delivery_days, delay_days.
Sorties	Une erreur de reconstruction par commande + un drapeau anomalie (au-dela d'un seuil).
Hyperparametres a regler	Taille du goulot d'etranglement (8), taille des couches, activation, optimiseur/learning_rate, batch_size, epoques + EarlyStopping, seuil (95e percentile).
Appris automatiquement	Les poids de l'encodeur et du decodeur.

6. GAN (generation de donnees synthetiques)

Generer des profils clients synthetiques realistes (augmentation, partage sans exposer les vraies donnees).

Ce qu'on attend	Produire des donnees clients artificielles mais realistes.
Entrees	Pour le generateur : un vecteur de bruit aleatoire . Pour le discriminateur : des vecteurs RFM reels standardises.
Caracteristiques utilisees	recency, frequency, monetary standardisees.
Sorties	Des echantillons synthetiques (memes colonnes) ; on compare les distributions reel/synthetique.
Hyperparametres a regler	Dimension du bruit, architectures generateur/discriminateur, Adam (lr=2e-4, beta1=0,5), LeakyReLU/BatchNorm/Dropout, batch_size, nb d'iterations, equilibre G/D .
Appris automatiquement	Les poids du generateur et du discriminateur.

7. GNN (systeme de recommandation)

Recommander des produits via un graphe d'interactions clients-produits (prediction de liens).

Ce qu'on attend	Recommander a un client des produits qu'il n'a pas encore achetes.
Entrees	Un graphe biparti (noeuds = clients + produits, aretes = achats) et sa matrice d'adjacence normalisee.
Caracteristiques utilisees	Les interactions client-produit (order_items + orders + customers) ; caracteristiques de noeuds (identite ou attributs).
Sorties	Des embeddings de noeuds et un score de lien (client, produit) donnant un top-N de recommandations.
Hyperparametres a regler	Nb de couches GCN (2), dimensions cachees/embedding, normalisation de l'adjacence, learning_rate, L2 , echantillonnage negatif , epoques ; (echelle) PyG / Spektral.
Appris automatiquement	Les matrices de poids de chaque couche du graphe.

Annexe - Reperes utiles

Parametre ou hyperparametre ?

Un **parametre** est **appris automatiquement** pendant l'entrainement (coefficients d'une regression, poids d'un reseau, centroides d'un K-means, seuils d'un arbre) : vous ne le fixez pas. Un **hyperparametre** est **fixe avant** l'entrainement et doit etre regle par vous (taux d'apprentissage, profondeur, nombre d'arbres, force de regularisation, taille des couches...). En ML, on les regle avec **GridSearchCV** ou **RandomizedSearchCV** + validation croisee ; en DL, on surveille les **courbes de validation** et on s'appuie sur les callbacks.

Quelle metrique pour quelle tache

Tache	Metriques recommandees
Classification (desequilibree)	Rappel, Precision, F1, ROC-AUC + matrice de confusion (l'accuracy seule trompe).
Regression	R^2 , MAE, RMSE + analyse des residus.
Segmentation	Score de silhouette, inertie, taille et interpretation des segments.
Serie temporelle	MAE / RMSE, compares a une reference naive.
Detection d'anomalies	ROC-AUC (independante du seuil), precision@k.
Recommandation (liens)	AUC, precision@k / rappel@k.

Bonne demarche (a rappeler aux etudiants)

1) Toujours **justifier** un hyperparametre par un critere (silhouette, courbe des k-distances, compromis biais/variance) et non par une valeur "magique". 2) Choisir la **metrique selon le besoin metier** (ici, privilegier le rappel sur les clients mecontents). 3) Comparer chaque modele a une **reference simple** (regression logistique / lineaire / prevision naive). 4) Sur donnees **tabulaires**, le Gradient Boosting rivalise souvent avec un reseau de neurones : le Deep Learning n'est pas toujours le meilleur choix — il brille sur l'image, le texte, les sequences et les graphes.

Ce guide se lit avec les notebooks fournis : *phase2_preparation_donnees.ipynb*, *phase4_machine_learning.ipynb* et *phase5_deep_learning.ipynb*, ou chaque choix d'hyperparametre est justifie dans le detail.